

Logistic Regression: Baseball or Hockey

Orobosa Ighinovia | COMP5420 | Assignment 3

I used logistic regression with stochastic gradient updates to predict baseball (1) or hockey (0). All experiments used seed 1701, the same 1,064 training documents and 133 held-out documents, and ten passes. The baseline features were word counts plus a bias. The learning rate controls the size of each weight update: smaller rates learn more slowly, while larger rates can fluctuate.

After ten passes, word-count models with learning rates 0.01, 0.1 and 0.5 reached 94.74%, 94.74% and 93.23% held-out accuracy, with mean log losses of 0.185, 0.145 and 0.238. Lazy L2 (step 0.1, $\mu = 0.001$) reached 93.98% accuracy and 0.173 loss; the decreasing rate reached 94.74% and 0.297. TF-IDF at step 0.1 reached 96.24% accuracy and 0.322 loss. All losses exclude the L2 penalty.

After one pass, rates 0.01, 0.1 and 0.5 gave 91.73%, 93.98% and 93.23% held-out accuracy. Rate 0.1 had the lowest final loss. Its training accuracy reached 99.91% at pass four and stayed there. Held-out accuracy was 94.74% from pass two onward except for 95.49% at pass five. About four passes were enough for accuracy to settle, although weights and loss continued changing.

For extra credit, the decreasing rate was $0.1 / \sqrt{t}$, where t is the update number starting at 1; training accuracy reached 95.58%. Lazy L2 delayed decay for inactive features and applied remaining decay before evaluation. It reached 96.24% held-out accuracy in passes three through seven, but the gain did not last. It still scans all features, so I did not establish a speed advantage.

For TF-IDF, I used $tf = 1 + \ln(\text{count})$ for present words and $idf = 1 + \ln((N + 1) / (df + 1))$, with $N = 1,197$ and df from the supplied vocabulary. Absent words have value zero. I multiplied tf by idf , divided the word vector by its L2 length, and kept the bias at 1. Training and evaluation both use this vector. At step 0.1, TF-IDF reached 99.15% training and 96.24% held-out accuracy. Its loss of 0.322 was higher than the count baseline: better classification accuracy did not mean better probability predictions. Normalization also changes feature scale, so the same step size need not be optimal.

To explain the baseline predictions, I ranked coefficients after ten passes at step 0.1, excluding the bias. The strongest baseball words were "runs" (+1.815), "pitching" (+1.523) and "baseball" (+1.217). The strongest hockey words were "hockey" (-2.925), "playoffs" (-1.996) and "golchowy" (-1.547). Mathematically, $\log(p / (1 - p)) = b + \sum(w_j x_j)$: each extra word occurrence changes baseball log-odds by its coefficient, holding other counts fixed. Ranking training-observed words by absolute coefficient identifies weak predictors; "dive," "privately" and "kills" had weights approximately zero.

These features ignore word order and context. All comparisons reuse one small held-out split; one document changes accuracy by about 0.75 percentage points. TF-IDF uses the supplied corpus frequencies, which include held-out documents; a strictly isolated evaluation would fit IDF on training documents only. The accuracy differences therefore deserve caution.